

Unit 5

Lists and Tuples



INTRODUCTION

- Sequence is a datatype that represents a group of elements.
- The purpose of any sequence is to store and process a group of elements. In Python, strings, lists, tuples and dictionaries are very important sequence datatypes.
- All sequences allow some common operations like indexing and slicing.
- **list** and **tuple** are *built in classes* in python
- In this chapter, you will learn about lists, tuples and their operations in Python.



LIST

- A list is similar to an array that consists of a group of elements or items.
- But, there is one major difference between an array and a list is, an array can store only one type of elements whereas a list can store different types of elements.
- Hence lists are more versatile and useful than an array.
- Perhaps lists are the most used datatype in Python programs.
- For example to store the student's information, we can create a list as:

```
student = [10, 'venu gopal', 'M', 50, 55, 62, 74, 66]
```

- Please observe that the elements of the 'student' list are stored in square braces [].
- we can create an empty list without any elements by simply writing empty square braces as:

```
e_lst = [ ] # this an empty list
```

- [Go to Jupyter notebook for examples](#)



CREATING LISTS USING RANGE() FUNCTION

- We can use range() function to generate a sequence of integers which can be stored in a list. The format of the range() function is:

range(start, stop, stepsize)

- If we do not mention the 'start', it is assumed to be 0 and the 'stepsize' is taken as 1.
- The range of numbers stops one element prior to 'stop'.
- [Go to Jupyter notebook for examples](#)



UPDATING THE ELEMENTS OF A LIST

- Lists are *mutable*. It means we can modify the contents of a list. We can append, update or delete the elements of a list depending upon our requirements.
- **Appending** an element means adding an element at the end of the list. To append a new element to the list, we should use the `append()` method
- **Updating** an element means changing the value of the element in the list. This can be done by accessing the specific element using *indexing or slicing and assigning a new value*
- **Deleting** an element from the list can be done using 'del' statement. The del statement takes the position number of the element to be deleted. We can also delete an element using the `remove()` method. In this method, we should pass the element to be deleted.
- To retrieve the elements of a list in **reverse** order we can use the `reverse()` method. This will reverse the order of elements in the list and the reversed elements are available in the list.



CONCATENATION OF TWO LISTS

- We can simply use '+' operator on two lists to join them.
- For example, 'x' and 'y' are the lists. If we write x+y, the list 'y' is joined at the end of the list 'x'

```
>>> x=[10,20,30,40,50]
>>> y=[100,110,120]
>>> print(x+y)
[10, 20, 30, 40, 50, 100, 110, 120]
```

REPETITION OF LISTS

- We can repeat the elements of a list 'n' number of times using '*' operator.
- For example, if we write x*n, the list 'x' will be repeated for n times as:

```
>>> x=[10,20,30,40,50]
>>> print(x*2)    # repeat the list x for 2 times
[10, 20, 30, 40, 50, 10, 20, 30, 40, 50]
```



MEMBERSHIP IN LISTS

- We can check if an element is a member of a list or not by using 'in' and 'not in' operator.
- If the element is a member of the list, then 'in' operator returns True else False. If the element is not in the list, then 'not in' operator returns True else False. See the examples below:

```
>>> x = [10,20,30,40,50]
```

```
>>> a = 20
```

```
>>> print(a in x)    # check if a is member of x
```

```
True
```

- If you write,

```
>>> print(a not in x)    # check if a is not a member of x
```

```
False
```



METHODS TO PROCESS LISTS

Method	Example	Description
sum()	list.sum()	Returns sum of all elements in the list.
index()	list.index(x)	Returns the first occurrence of x in the list
append()	list.append(x)	Appends x at the end of the list
insert()	list.insert(i, x)	Inserts x in to the list in the position specified by i
copy()	list.copy()	Copies all the list elements into a new list and returns it
extend()	list.extend(list1)	Appends list1 to list
count()	list.count(x)	Returns number of occurrences of x in the list
remove()	list.remove(x)	Removes x from the list
pop()	list.pop()	Removes the ending element from the list
sort()	list.sort()	Sorts the elements of the list into ascending order
reverse()	list.reverse()	Reverses the sequence of elements in the list
clear()	list.clear()	Deletes all elements from the list

- [Go to Jupyter notebook for examples](#)



SORTING THE LIST ELEMENTS

- Python provides the `sort()` method to sort the elements of a list. This method can be used as:

```
x.sort()
```

- This will sort list 'x' into ascending order
- If we want to sort the elements of the list into descending order, then we can mention 'reverse=True' as a parameter in the `sort()` method as:

```
x.sort(reverse=True)
```

- This will sort the list 'x' into descending order.
- We can also sort elements of the list programmatically
- [Go to Jupyter notebook for examples](#)



TUPLES

- A tuple is a Python sequence which stores a group of elements or items.
- Tuples are similar to lists but the main difference is tuples are immutable whereas lists are mutable.
- Since tuples are immutable, once we create a tuple we cannot modify its elements. Hence we cannot perform operations like `append()`, `extend()`, `insert()`, `remove()`, `pop()` and `clear()` on tuples.
- Tuples are generally used to store data which should not be modified and retrieve that data on demand.



CREATING TUPLES

- We can create a tuple by writing elements separated by commas inside parentheses ().
- The elements can be of same datatype or different types.
- For example, to create an empty tuple, we can simply write empty parenthesis, as:

```
tup1 = ()    # empty tuple
```

- If we want to create a tuple with only one element, we can mention that element in parentheses and after that a comma is needed, as:

```
tup2 = (10,) # tuple with one element. Observe comma after the element.
```

- Here is a tuple with different types of elements:

```
tup3 = (10, 20, -30.1, 40.5, 'Hyderabad', 'New Delhi')
```

- We can create a tuple with only one of type of elements also, like the following:

```
tup4 = (10, 20, 30)    # tuple with integers
```

- If we do not mention any brackets and write the elements separating them by commas, then they are taken by default as a tuple.
- See the following example:

```
tup5 = 1, 2, 3, 4    #no braces
```



- It is also possible to create a tuple from a list. This is done by converting a list into a tuple using the tuple() function. Consider the following example:

```
list = [1, 2, 3]    # take a list  
tpl = tuple(list)   # convert list into tuple  
print(tpl)          # display tuple
```

- The tuple is shown below:

```
(1, 2, 3)
```

- Another way to create a tuple is by using range() function that returns a sequence.
- To create a tuple 'tpl' that contains numbers from 4 to 8 in steps of 2

```
tpl= tuple(range(4, 9, 2))  
print(tpl)
```

- The preceding statements will give: (4, 6, 8)



ACCESSING THE TUPLE ELEMENTS

- Accessing the elements from a tuple can be done using indexing or slicing. This is same as that of a list.
- [Go to Jupyter notebook for examples](#)

BASIC OPERATIONS ON TUPLES

- The 5 basic operations: finding length, concatenation, repetition, membership and iteration operations can be performed on any sequence may be it is a string, list, tuple or a dictionary.
- [Go to Jupyter notebook for examples](#)



FUNCTIONS TO PROCESS TUPLES

Function	Example	Description
len()	len(tpl)	Returns the number of elements in the tuple.
min()	min(tpl)	Returns the smallest element in the tuple.
max()	max(tpl)	Returns the biggest element in the tuple.
count()	tpl.count(x)	Returns how many times the element 'x' is found in tpl.
index()	tpl.index(x)	Returns the first occurrence of the element 'x' in tpl. ValueError if 'x' is not found in the tuple.
sorted()	sorted(tpl)	Sorts the elements of the tuple into ascending order. sorted(tpl, reverse=True) will sort in reverse order.

- [Go to Jupyter notebook for examples](#)



NESTED TUPLES

- A tuple inserted inside another tuple is called nested tuple.
- For example, `tup = (50,60,70,80,90, (200, 201))` # tuple with 6 elements
- Observe the last parentheses in the tuple `tup`, i.e. `(200, 201)`. These parentheses represent that it is a tuple with 2 elements inserted into the tuple `'tup'`. The tuple `(200, 201)` is called nested tuple as it is inside another tuple.

```
print('Nested tuple= ', tup[5])
```

- The preceding statement will give the following output:

```
Nested tuple= (200, 201)
```

- Every nested tuple can represent a specific data record. For example, to store 4 employees data in `'emp'` tuple, we can write:

```
emp = ( (10, "Vijay", 9000.90), (20, "Nihaar", 5500.75), (30, "vanaja", 8900.00), (40, "Kapoor", 5000.50) )
```

- Here, `'emp'` is the tuple name. It contains 4 nested tuples each of which represents the data of an employee. Every employee's identification number, name and salary are stored as a nested tuples.



SORTING NESTED TUPLES

- To sort a tuple, we can use `sorted()` function. This function sorts by default into ascending order.
- *[Go to Jupyter notebook for examples](#)*



INSERTING ELEMENTS IN A TUPLE

- Since tuples are immutable, we cannot modify the elements of the tuple once it is created.
- Now, let's see how to insert a new element into an existing tuple.
- [Go to Jupyter notebook for examples](#)

MODIFYING ELEMENTS OF A TUPLE

- It is not possible to modify or update an element of a tuple since tuples are immutable.
- If we want to modify the element of a tuple, we have to create a new tuple with a new value in the position of the modified element
- [Go to Jupyter notebook for examples](#)



DELETING ELEMENTS FROM A TUPLE

- The simplest way to delete an element from a particular position in the tuple is to copy all the elements into a new tuple except the element which is to be deleted.
- *[Go to Jupyter notebook for examples](#)*

